TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO

HỌC PHẦN: TRÍ TUỆ NHÂN TẠO NÂNG CAO

EX01: TÌM KIẾM TRONG AI

Giảng viên hướng dẫn: Đỗ Như Tài
Sinh viên thực hiện: Trương Thị Huyền Trang – 3123410385

TP. Hồ Chí Minh, tháng 10 năm 2026

BÀI TẬP TÌM KIẾM TRONG AI

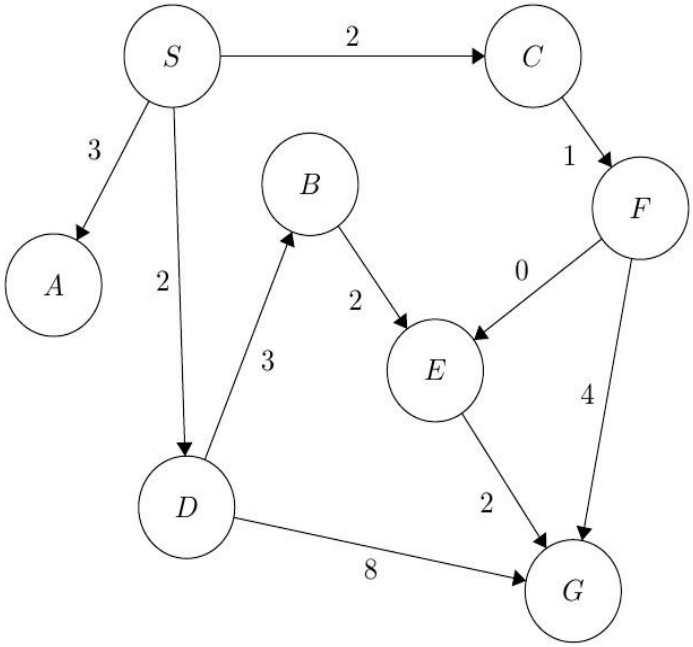
Họ và tên	Trương Thị Huyền Trang
MSSV / Lớp	3123410385

Mục tiêu học tập

- Mô phỏng các thuật toán BFS, DFS và Iterative Deepening.
- Biểu diễn bài toán Tháp Hà Nội dưới dạng bài toán tìm kiếm.
- Thiết kế và phân tích heuristic cho A*.
- So sánh DFS, BFS, UCS, Greedy Search và A* trên cùng một bài toán.
- Phân tích các phát biểu lý thuyết về heuristic, A*, Minimax và Expectimax.

Bài 1. Các thuật toán tìm kiếm

Cho đồ thị có trạng thái bắt đầu S và trạng thái đích G như hình dưới đây.



Hình 1. Đồ thị dùng cho các thuật toán tìm kiếm

Yêu cầu: Với mỗi thuật toán, hãy ghi thứ tự các nút được đưa vào explored set, đường đi được trả về và tổng chi phí của đường đi. Nếu có hòa, phá hòa theo thứ tự alphabet của trạng thái cuối cùng trong đường đi.

1.1. Breadth-First Search (BFS)

Nội dung	Kết quả
Explored Set	S, A, C, D, B, F, G
Path	S -> D -> G

Total Cost	2+8=10

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	S, A, C, F, E, G
Path	S -> C -> F -> E ->G
Total Cost	2+1+0+2=5

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S, A, C, D
2	S, A, C, F, D, B, G
...	

Path cuối cùng: S->D->G

Total Cost: 2+8=10

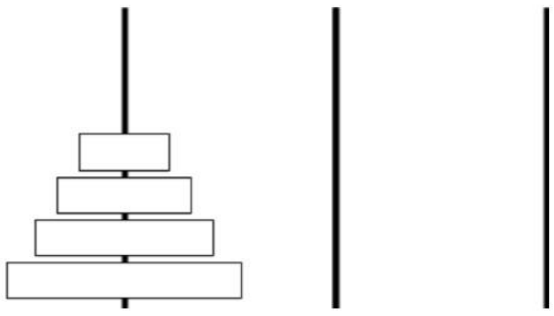
Câu hỏi thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

BFS: frontier hoạt động gần giống queue – FIFO, mở rộng node có độ sâu nhỏ trước, BFS complete và tối ưu về số bước nếu chi phí mỗi cạnh bằng nhau.

DFS: frontier hoạt động gần giống stack – LIFO, ưu tiêu đi sâu trước. DFS dùng ít bộ nhớ hơn nhưng không đảm bảo tìm được đường đi tối ưu.

Bài 2. Tháp Hà Nội - Tower of Hanoi

Bài toán gồm n đĩa có kích thước khác nhau và ba cọc. Ban đầu các đĩa được xếp trên một cọc theo thứ tự từ lớn đến nhỏ. Mỗi lần chỉ được di chuyển một đĩa và không được đặt đĩa lớn lên trên đĩa nhỏ. Mục tiêu là chuyển toàn bộ đĩa sang cọc thứ ba.



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội

2.1. Biểu diễn trạng thái

Hãy đề xuất cách biểu diễn một trạng thái của bài toán. Biểu diễn phải cho biết mỗi đĩa đang ở cọc nào và đủ để xác định các hành động hợp lệ.

Biểu diễu bằng một vector: state=(p1, p2, …, pn)
Trong đó: $pi \in \{1, 2, 3\}$ là số hiệu của các cọc i chứa đĩa.

Ví dụ với 4 đĩa:

- (1, 1, 1, 1) có nghĩa là cả 4 cọc đĩa đều đang ở cọc 1.
- (3, 2, 1, 1) nghĩa là: đĩa 1 ở cọc 3, đĩa 2 ở cọc 2, đĩa 3 và đĩa 4 đều ở cọc 1.

Quy ước đĩa 1 là nhỏ nhất.

2.2. Kích thước không gian trạng thái

Với n đĩa, hãy xác định số trạng thái có thể có theo cách biểu diễn của bạn và giải thích cách suy luận.

Mỗi đĩa có thể nằm trên 1 trong 3 cọc. Với n dĩa có: 3^n trạng thái. Lý do là thứ tự các đĩa trên cùng một cọc đã được xác định với kích thước: đĩa nhỏ luôn phải nằm trên đĩa to hơn.

2.3. Trạng thái bắt đầu

Hãy viết trạng thái khởi đầu S0 theo cách biểu diễn đã chọn.

Ban đầu, tất cả các đĩa đều nằm ở cọc 1: S0 = (1,1,1,1)

2.4. Các hành động hợp lệ

Từ một trạng thái bất kỳ, hãy mô tả tập Actions(s). Một hành động hợp lệ phải thỏa mãn các điều kiện nào?

Mỗi hành động có dạng : Move(i, A, B)

Nghĩa là di chuyển đĩa i từ cọc A sang cọc B.

Hành động hợp lệ khi:
1. Đĩa i nằm trên cùng của cọc A.
2. Cọc B đang trống hoặc đĩa trên cùng của cọc B lớn hơn đĩa i.
3. Mỗi lần chỉ được di chuyển một đĩa.

2.5. Goal Test

Hãy mô tả hàm goalTest(state): khi nào hàm trả về True?

Hàm trả về True khi toàn bộ các đĩa đã được chuyển sang cọc thứ ba.

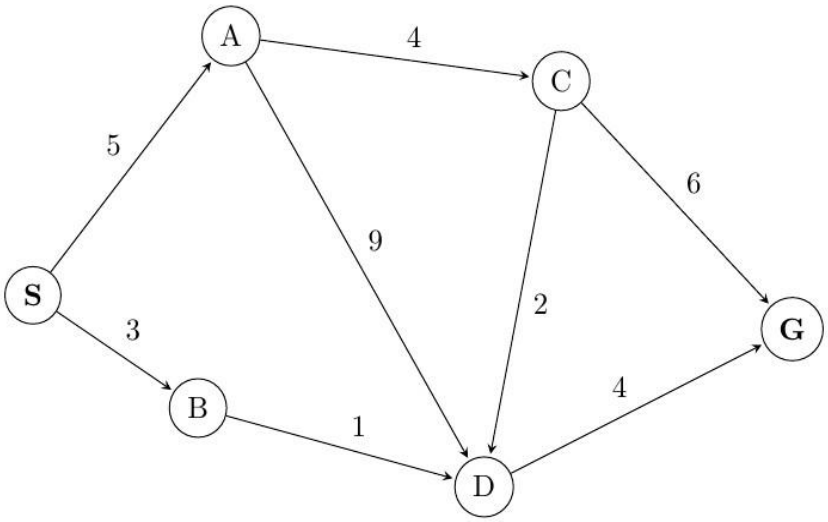
goalTest(state) = true :

state=(3,3,3,….,3)

Với 4 đĩa: (3,3,3,3)

Bài 3. Thiết kế và phân tích Heuristic

Xét đồ thị dưới đây. Hãy thiết kế heuristic và phân tích ảnh hưởng của heuristic đến thuật toán A*.



Hình 3. Đồ thị dùng để thiết kế heuristic

Công thức A*: f(n) = g(n) + h(n), trong đó g(n) là chi phí thực tế từ S đến n và h(n) là chi phí ước lượng từ n đến đích.

3.1. Thiết kế heuristic admissible và consistent

Điền giá trị h(n) cho từng nút sao cho heuristic phù hợp với yêu cầu của nhóm.

Node	h(n)
S	8
A	10
B	5
C	6
D	4
G	0

3.2. Chạy A* với heuristic đã thiết kế Explored

Set:

S: g(S) = 0

h(S)=8

f(S) = 0 + 8 = 8

Mở rộng S:

S->A:

g(A) = 5

h(A) = 10

f(A) = 5 + 10 = 15
S->B:

g(B) = 3

h(B) = 5

f(B) = 3+5=8

⇨ Chọn A

Từ B:

B -> D

g(D) = 3+1=4

h(D)=4

f(D) = 8

⇨ Chọn D

Từ D:

D->G

g(G) = 4+4=8

h(G) = 0

f(G)=8

⇨ Chọn G => Đạt mục tiêu
⇨ Explored set: S, B, D, G

Path: S -> B -> D → G

Total Cost: 3+1+4 = 8

3.3. Heuristic admissible nhưng không consistent

Đề xuất một bộ giá trị h(n) admissible nhưng không consistent. Chỉ ra ít nhất một cạnh vi phạm điều kiện h(n) <= c(n,n') + h(n').

Ta có thể chọn:

Node	h(n)
S	8
A	10
B	5
C	6
D	**0**
G	0

Các giá trị này vẫn không vượt chi phí thực đến G, nên vẫn admissible.
Xét B -> D:

Có cost 1.
Consistency yêu cầu: h(B) <=c(B,D)+h(D)

Thay số: 5 <= 1+ 0 (SAI)

Vì vậy: heuristic trên admissible nhưng không consistent.

3.4. Câu hỏi mở rộng

Giải thích vì sao một heuristic consistent thì cũng admissible, giả sử h(G) = 0.

Nếu heuristic consistent thì với mọi cạnh: h(n) <= c(n,n')+h(n')

Giả sử đường tối ưu: n->n1->n2->…→G

Ta có: h(n)<=c(n,n1)+h(n1)

h(n1)<=c(n1,n2)+h(n2)

…

Cuối cùng: h(G) = 0

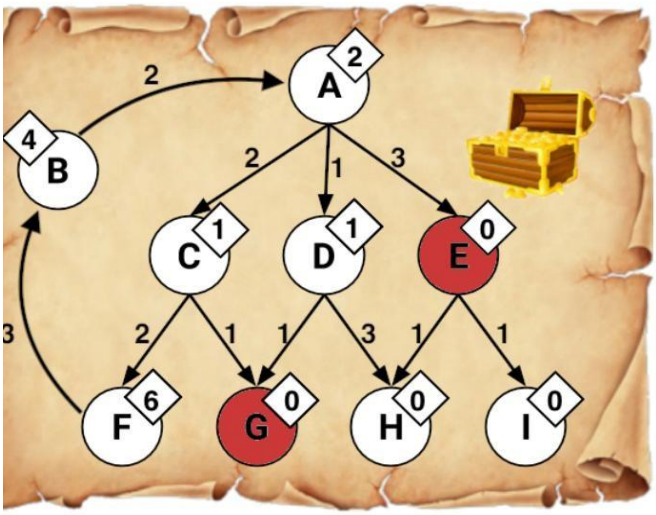
Suy ra: h(n)<= c(n,n1)+c(n1,n2)+…+c(nk,G)

Vế phải chính là chi phí tối ưu thực tế: h*(n)

⇨ h(n)<=h*(n)
⇨ Heuristic là addmissible.

Bài 4. Treasure Hunting (Tìm kho báu)

A là đảo xuất phát. Các nút màu đỏ là các đảo có kho báu; chỉ cần tới một trong các đảo đó. Mũi tên biểu diễn hành động có hướng; số trên cạnh là action cost. Giá trị trong hình thoi là heuristic ước lượng chi phí tối thiểu từ nút hiện tại tới một kho báu.



Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)

Quy tắc phá hòa: khi lấy phần tử khỏi frontier, nếu hòa thì chọn theo thứ tự alphabet.

4.1. Depth-First Search Explored

Set:

*Tại A:

C, D, E => Chọn C

Từ C: F, G => Chọn F

F -> B

B->A nhưng A đã explored nên quay lại C -> G.

G là kho báu.

Explored set: A, C, F, B, G

Path returned: A-> C->G

4.2. Breadth-First Search Explored

Set:

Từ A: Các nút depth 1: C, D, E

Theo Alphabet: C->D->E.

E là kho báu.

Explored set: A, C, D, E

Path returned: A->E

4.3. Uniform-Cost Search Explored

Set:

Chọn node có g(n) nhỏ nhất.

*Từ A:

Chọn D : g(D)= 1

D tạo :

G: g=1+1=2

H: g=1+3=4

Frontier:

C=2

G=2

E=3

H=4

C và G hòa, chọn thep alphabet => Chọn C.

Từ C:

F: g=4

G: g=3

Nhưng đã có G với g=2, giữ đường tốt hơn.

Chọn G.

Explored Set: A, D, C, G

Path returned: A->D->G

Cost: 1+1=2

4.4. Greedy Search Explored

Set:

Greedy chọn node có h(n) nhỏ nhất.

*Từ A:

Chọn E (kho báu)

Explored set: A,E

Path returned: A->E

Cost: 3

4.5. A* Search Explored

Set:

A* chọn: f(n)=g(n)+h(n)

Từ A:

C: g=2, h=1, f=3

D: g=1, h=1, f=2

E: g=3, h=0, f=3

Chọn D.

Từ D:

G: g=1+1=2, f=2+0=2

H: g=1+3=4, f=4

Chọn G (Kho báu)

Path returned: A->D->G

Cost: 2

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu	A->C->G	3
BFS	Độ sâu nhỏ nhất	A->E	3
UCS	g(n)	A->D->G	2
Greedy	h(n)	A->E	3
A*	g(n)+h(n)	A->D->G	2

Bài 5. Câu hỏi Đúng/Sai

Với mỗi phát biểu, hãy ghi Đúng hoặc Sai và giải thích ngắn gọn. Nếu sai, nên đưa ra phản ví dụ.

Câu 1. Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic. Đúng / Sai: **SAI**

Giải thích:

DFS đôi khi có thể may mắn đi đúng nhánh chứa goal ngay và mở rộng rất ít node, trong khi A* có thể phát xét thêm nhiều node khác. Vì vậy không thể khẳng định DFS luôn mở rộng >=A*.

Câu 2. Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B. Đúng / Sai: **SAI**

Giải thích:

Quân xe có thể đi nhiều ô theo hàng hoặc cột chỉ trong một nước đi. Ví dụ A và B nằm cùng hàng, cách nhau 5 ô:

Manhattan distance = 5. Nhưng quân xe chỉ cần 1 nước đi .

Do đó: h=5 > h*=1. Nên không admissible.

Câu 3. Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman. Đúng / Sai: **ĐÚNG**

Giải thích:

Với bài toán Pacman dạng lưới chuẩn, mỗi bước có cost 1. Khoảng cách Euclidean không lớn hơn chiều dài đường thực tế: Euclidean(n,G)<=h*(n).

Đường đi thực tế còn có thể dài hơn vì có tường/chướng ngại vật.

Câu 4. Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

Đúng / Sai: **SAI**

Giải thích:

Ví dụ: h1(n) = 5, h2(n)=6 và chi phí thực h*(n)=6.

Riêng từng heuristic: 5<=6 nên admissible.

Nhưng h1+h2 = 10 > 6 nên tổng không admissible.
Thông thường max(h1,h2) vẫn admissible nếu cả hai đều admissible.

Câu 5. Admissibility của một heuristic cho A* kéo theo consistency.

Đúng / Sai: **SAI**

Giải thích: Consistency là điều kiện mạnh hơn admissbility.

Câu 6. A* với graph search luôn tối ưu nếu heuristic admissible.

Đúng / Sai: **Sai**

Giải thích:

Sai nếu graph search dùng closed set và không mở lại node. Để A* graph search đảm bảo tối ưu trong cách triển khai thông thường, heuristic cần consistent. Chỉ admissible nhưng không consistent có thể làm một node bị đóng bằng đường đi chưa tốt nhất.

Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uninformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	g(n)
Informed Search	Greedy	h(n)
Informed Search	A*	g(n)+h(n)

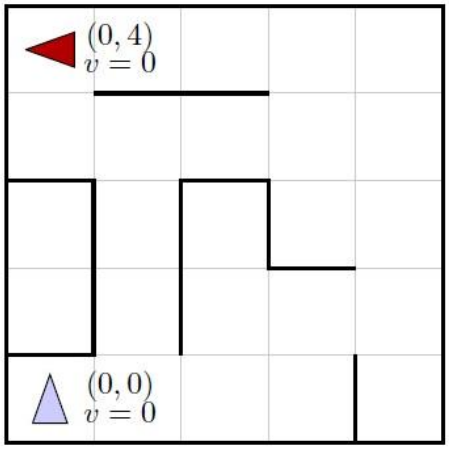
Ghi nhớ: A* dùng f(n) = g(n) + h(n).

BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Mục tiêu học tập

- Mô hình hóa bài toán tìm kiếm trong AI.
- Vận dụng BFS, UCS, Greedy và A*.
- Phân tích và đánh giá heuristic.
- Phân biệt heuristic admissible và consistent.
- So sánh tính đầy đủ, tối ưu và hiệu quả của các thuật toán tìm kiếm.

Câu 1. Tìm kiếm và heuristic



Một tác tử dạng xe cần thoát khỏi mê cung như hình trên. Tác tử có hướng và tại mọi thời điểm luôn quay về một trong bốn hướng d ∈ {Bắc, Nam, Đông, Tây}. Trong mỗi bước, tác tử có thể tiến về phía trước với vận tốc có thể điều chỉnh hoặc thực hiện thao tác rẽ.

Các hành động di chuyển gồm tăng tốc, giữ tốc độ và giảm tốc. Sau hành động, tác tử di chuyển một số ô bằng vận tốc mới v'. Gọi v là vận tốc hiện tại và v' là vận tốc sau điều chỉnh:

- Tăng tốc (Faster): v' = v + 1
- Giảm tốc (Slower): v' = v - 1
- Giữ tốc độ (Maintain): v' = v

Các hành động rẽ gồm trái và phải, làm thay đổi hướng 90°. Chỉ được phép rẽ khi vận tốc bằng 0; sau khi rẽ vận tốc vẫn bằng 0.

- Rẽ trái: quay 90° ngược chiều kim đồng hồ.
- Rẽ phải: quay 90° theo chiều kim đồng hồ.

Ví dụ: nếu tác tử ở (0,0), quay về Bắc, v = 0 và cần đến (2,0), quay về Đông, v = 0 thì một chuỗi hành động hợp lệ là: rẽ phải, tăng tốc, giữ tốc độ, giảm tốc.

Các hành động không hợp lệ gồm:

- Hành động làm tác tử va vào tường.
- Làm v < 0 hoặc v > Vmax.
- Giữ tốc độ khi v = 0.

- Rẽ khi $v \neq 0$.

Mục tiêu là tìm kế hoạch đưa xe dừng hẳn ($v = 0$) tại ô thoát, đúng hướng đích, với số hành động ít nhất. Chi phí đường đi được tính bằng số hành động đã thực hiện.

a) Giả sử tác tử đi theo nhánh ngoài cùng bên trái (đi qua ô (1,2)), từ trạng thái bắt đầu (0,0), hướng Bắc đến trạng thái đích (0,4), hướng Tây. Hãy viết chuỗi hành động ngắn nhất.

Rẽ phải -> Faster -> Slower -> Rẽ trái-> Faster -> Maintain -> Maintain -> Slower -> Rẽ trái -> Faster -> Slower -> Rẽ phải -> Faster -> Slower -> Rẽ trái

b) Nếu lưới có kích thước M × N và vận tốc tối đa là Vmax, kích thước không gian trạng thái là bao nhiêu? Giả sử mọi cấu hình đều có thể đạt được từ trạng thái bắt đầu.

Mỗi trạng tháng gồm 4 thành phần: (x, y, hướng, vận tốc). Có:

- (M*N) vị trí
- 4 hướng
- Vận tốc từ 0 đến Vmax, tức có Vmax+1 giá trị

Do đó: M*N*4*(Vmax+1)
Ví dụ nếu: M=5, N=5, Vmax=3
Thì: 5*5*4*4=400 trạng thái

c) Một "nút con" của trạng thái s là trạng thái s' có thể đạt được bằng một hành động hợp lệ từ s. Có thể tồn tại trạng thái không có nút con hay không? Nếu có, hãy cho ví dụ; nếu không, hãy giải thích.

✓ **Có** ☐ Không

Một nút con là trạng thái đạt được sau một hành động hợp lệ.
Ví dụ xe:

- Đang hướng thẳng vào tường,
- v=2
- Không được rẽ vì v khác 0.

Khi đó:
Faster -> đâm tường
Maintain -> đâm tường
Slower -> v'=1 nhưng vẫn đi vào tường
Rẽ trái -> không được vì $v \neq 0$
Rẽ phải -> không được vì $v \neq 0$
⇨ Không có hành động hợp lệ => Không có nút con.

d) Hệ số phân nhánh lớn nhất của bài toán là bao nhiêu? Hãy cho một trạng thái ví dụ (x, y, hướng, vận tốc) đạt hệ số phân nhánh này và liệt kê các hành động có thể thực hiện.

Hệ số phân nhánh = số hành động hợp lệ tối đa từ một trạng thái.
Nếu: v=0 , ta có thể: Faster, rẽ trái, rẽ phải.

Maintain khi v=0 không hợp lệ và Slower làm vận tốc âm nên cũng không hợp lệ.
Vậy: bmax = 3.

e) Khoảng cách Manhattan từ vị trí hiện tại của tác tử đến vị trí cửa ra có phải là heuristic chấp nhận được không? Nếu không, hãy cho một trạng thái phản ví dụ, nêu giá trị heuristic và chi phí thực tế tối ưu đến đích.

☐ Có ✔ **Không**

Manhattan Distance: $|x1 - x2| + |y1 - y2|$
Nó đo số ô, nhưng chi phí trong bài này lại là số hành động. Xe có thể đi nhiều ô chỉ trong một hành động nếu vận tốc lớn.
Ví dụ giả sử còn cách mục tiêu 6 ô và xe có vận tốc phù hợp. Xe có thể đi:
3 ô
2 ô
1 ô
Chỉ với vài hành động thay vì 6 hành động.
Khi đó có thể xảy ra: h(n)=6.
Nhưng: h*(n)<6
Tức heuristic đánh giá cao hơn chi phí thật, vi phạm điều khiện: h(n) <= h*(n) . Nên Manhattan không admissible.

f) Heuristic h = khoảng cách Manhattan / Vmax có phải là admissible không? Giải thích. Nếu không, hãy cho phản ví dụ tương tự câu (e).

✔ **Có** ☐ Không

Vì trong một hành động di chuyển, xe nhiều nhất chỉ có thể đi Vmax ô.
Nếu Manhattan Distance là D, thì tối thiểu ta cần khoảng: (D/Vmax) hành động di chuyển.
Trong thực tế còn có thể phải: tăng/giảm tốc độ, dừng, rẽ, né tường. Do đó chi phí thật thường lớn hơn hoặc bằng (D/Vmax).
Vậy: h(n) <= h*(n). Nên heuristic này admissible.

g) Nếu dùng heuristic không admissible trong A* Tree Search, tính đầy đủ của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☐ Có ✔ **Không**

Trong điều kiện của bài: đồ thị hữu hạn, heuristic không âm, mỗi hành động có chi phí dương.
Heuristic không admissible có thể làm A* chọn node không tốt, nhưng nếu có lời giải thì A* vẫn có khả năng tìm được.
Vì vậy completeness không nhất thiết bị mất.

h) Nếu dùng heuristic không admissible trong A* Tree Search, tính tối ưu của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

✔ **Có** ☐ Không

Nếu heuristic đánh giá quá cao: h(n) > h*(n).
A* có thể tránh một đường thực sự tốt vì tưởng rằng đường đó quá đắt. Kết quả là A* có thể trả về đường đi cost = 15. Trong khi tồn tại đường đi cost = 10. Vậy non-admissible heuristic có thể làm A* mất optimality.

i) Những lý do nào sau đây có thể hợp lý khi dùng heuristic không admissible thay vì admissible? Chọn tất cả phương án phù hợp.

✔ **Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.**

✔ **Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.**

☐ Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.

✔ **Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.**

Câu 2. Các nút tìm kiếm

Xét tree search (không dùng tập explored) cho một bài toán tìm kiếm bất kỳ có hệ số phân nhánh tối đa b. Mỗi nút tìm kiếm n có chi phí tích lũy g(n), heuristic admissible h(n) và độ sâu d(n). Gọi n_c là nút đích có chi phí nhỏ nhất và n_s là nút đích nông nhất.

Với mỗi trường hợp, hãy viết một bất đẳng thức mô tả tập các nút được mở rộng trước khi thuật toán kết thúc. Không cần xử lý trường hợp hòa (tie). Nếu không có nút nào thỏa, ghi "không có".

a) Breadth-First Search (BFS).

Bất đẳng thức: d(n) < d(ns)

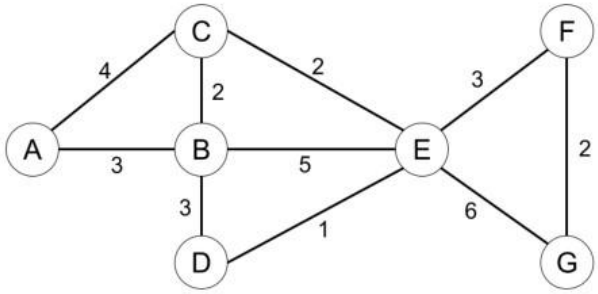
b) Uniform-Cost Search (UCS).

Bất đẳng thức: g(n) < g(nc)

c) A* Search, giả sử h là heuristic nhất quán.

Bất đẳng thức: g(n) + h(n) < g(nc) => f(n) < g(nc)

Câu 3. Tìm kiếm trên đồ thị



Node	h_1	h_2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

Nút	h1	h2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

Xét đồ thị vô hướng như hình. A là trạng thái bắt đầu và G là trạng thái đích. Chi phí của mỗi cạnh được ghi trên đồ thị. Bảng trên cung cấp hai heuristic h1 và h2. Khi có nhiều nút cùng ưu tiên, kết quả có thể phụ thuộc quy tắc phá hòa (tie-breaking).

a) Với mỗi chiến lược tìm kiếm đồ thị dưới đây, hãy đánh dấu X vào mọi đường đi mà thuật toán có thể trả về. Nếu kết quả không nằm trong các đường đi liệt kê, ghi đường đi đúng ở cột "Khác".

Thuật toán	A-C-E-G	A-C-E-F-G	A-B-D-E-F-G	Khác
UCS	☐	✔	☐	☐
Greedy với h1	☐	☐	☐	✔ A-B-E-G
Greedy với h2	✔	☐	☐	☐
A* với h1	☐	☐	✔	☐
A* với h2	☐	✔	☐	☐

b) Chi phí của đường đi tối ưu từ A đến G theo UCS là bao nhiêu?

Đáp án: A-C-E-F-G
Cost: 4+2+3+2 = 11

c) Heuristic h1 có admissible không? Có consistent không?

Admissible: ☐ Có ✔ Không Consistent: ☐ Có ✔ Không

d) Heuristic h2 có admissible không? Có consistent không?

Admissible: ✔ Có ☐ Không Consistent: ☐ Có ✔ Không

Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

a) Đánh dấu Đúng/Sai. Giả sử mọi hành động có chi phí ít nhất ε, với ε > 0, và mọi heuristic được dùng đều consistent.

Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☐ Đúng ✔ Sai

Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

✔ Đúng ☐ Sai

Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

✔ Đúng ☐ Sai

Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.

☐ Đúng ✔ Sai

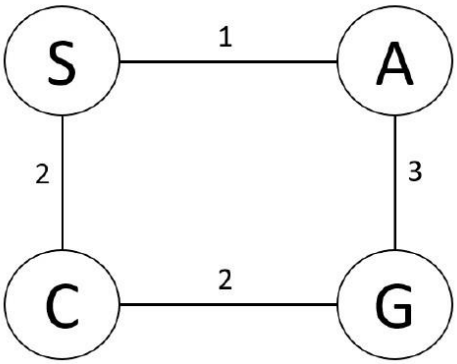
Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS.

☐ Đúng ✔ Sai

Nếu h1(s) và h2(s) đều là heuristic consistent thì min(h1(s), h2(s)) cũng nhất thiết consistent.

✔ Đúng ☐ Sai

b) Xét đồ thị trạng thái dưới đây, S là bắt đầu và G là đích. Hãy điền các giá trị heuristic sao cho heuristic admissible nhưng không consistent.



Heuristic Function	
State	$h(s)$
S	
A	
C	
G	0

Trạng thái	h(s) (Chi phí tối ưu thật)
S	4
A	3
C	2
G	0

Muốn admissible, chỉ cần không vượt các giá trị đó. Ta chọn:

Trạng thái	h(s)
S	4
A	**0**
C	2
G	0

Tất cả đều addmissible.

Xét cạnh: S-A có cost = 1. Consistency yêu cầu: h(S)<= 1+h(4) ➔ 4<=1 ➔ SAI.

Vậy bộ trên là admissible nhưng không consistent.

--- Hết ---